scripts/dispatch_work_orders.py

import csv import hashlib import hmac import json import os import sys import time from urllib.parse import urlparse
import requests
QUEUES = [("output/send_sol_notice.csv", "SOL Expungement Notices"), ("output/send_irs_complaint.csv", "501(r) Regulatory Complaints"), ("output/priority_queue.csv", "Certified Mail Buffer Warnings"),]
REQUIRED_COLUMNS = { "Account ID", "Document ID", "Violation Type", "Action Deadline", "Status", }
MAX_ATTEMPTS = 3
def required_env(name: str) -> str: value = os.getenv(name, "").strip() if not value: raise RuntimeError(f"{name} is required.") return value
def validate_endpoint(url: str) -> None: parsed = urlparse(url) if parsed.scheme != "https" or not parsed.netloc: raise RuntimeError("Dispatch endpoint must be a valid HTTPS URL.")
def load_rows(path: str) -> list[dict]: if not os.path.isfile(path): return []
with open(path, newline="", encoding="utf-8") as f: reader = csv.DictReader(f) if not reader.fieldnames: raise RuntimeError(f"{path}: CSV has no header.") missing = REQUIRED_COLUMNS - set(reader.fieldnames) if missing: raise RuntimeError(f"{path}: missing columns: {', '.join(sorted(missing))}") return list(reader)
def idempotency_key(secret: bytes, category: str, row: dict) -> str: identity = { "category": category, "account_id": row["Account ID"], "document_id": row["Document ID"], "violation_type": row["Violation Type"], "action_deadline": row["Action Deadline"], } canonical = json.dumps(identity, sort_keys=True, separators=(",", ":")) return hmac.new(secret, canonical.encode("utf-8"), hashlib.sha256).hexdigest()
def post_work_order(session: requests.Session, endpoint: str, secret: bytes, category: str, row: dict) -> None: key = idempotency_key(secret, category, row) payload = { "category": category, "account_id": row["Account ID"], "document_id": row["Document ID"], "violation_type": row["Violation Type"], "action_deadline": row["Action Deadline"], "status": row["Status"], }
for attempt in range(1, MAX_ATTEMPTS + 1): try: response = session.post(endpoint, json=payload, headers={"Idempotency-Key": key}, timeout=(5, 20), allow_redirects=False,) response.raise_for_status() return except requests.RequestException: if attempt == MAX_ATTEMPTS: raise time.sleep(2 (attempt - 1))
def main() -> int: endpoint = required_env("DISPATCH_WEBHOOK_URL") validate_endpoint(endpoint) secret = required_env("IDEMPOTENCY_KEY_SECRET").encode("utf-8")
sent = 0 failures = 0
with requests.Session() as session: for path, category in QUEUES: try: rows = load_rows(path) except (OSError, csv.Error, RuntimeError) as exc: print(f"Cannot read queue {path}: {exc}", file=sys.stderr) failures += 1 continue
for row in rows: try: post_work_order(session, endpoint, secret, category, row) sent += 1 except requests.RequestException as exc: # Sanitize error output: omit payload bodies to protect PHI/PII print(f"Dispatch failed for {category}, queue {path}: " f"{type(exc).**name**}", file=sys.stderr,) failures += 1
print(f"Dispatch finished: {sent} accepted, {failures} failed.") return 1 if failures else 0
if **name** == "**main**": sys.exit(main())

```yaml:GitHub Actions Dispatch Job:.github/workflows/dispatch.yml
name: Dispatch Work-Orders

on:
  push:
    branches:
      - main

jobs:
  dispatch:
    needs: audit
    if: >-
      always() &&
      github.event_name == 'push' &&
      github.ref == 'refs/heads/main' &&
      needs.audit.result != 'cancelled'
    runs-on: ubuntu-latest
    permissions:
      actions: read
      contents: read
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Download Audit Artifacts
        uses: actions/download-artifact@v4
        with:
          name: medical-debt-audit-artifacts
          path: output

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install Dependencies
        run: pip install requests==2.32.4

      - name: Dispatch Work-Orders to Internal Case API
        env:
          DISPATCH_WEBHOOK_URL: ${{ secrets.DISPATCH_WEBHOOK_URL }}
          IDEMPOTENCY_KEY_SECRET: ${{ secrets.IDEMPOTENCY_KEY_SECRET
}}
        run: python scripts/dispatch_work_orders.py
```eCFR

---

### Key Architectural Safeguards Enforced

1. **Deterministic Idempotency:** The HMAC-SHA256 signature calculated from the canonical `category`, `account_id`, `document_id`, `violation_type`, and `action_deadline` payload guarantees that retried HTTP requests are deduplicated by the receiving API without producing duplicate notices.
2. **Data Minimization & PII Boundary:** Row-level identifiers, patient account numbers, and document checksums are transmitted **only** to the encrypted HTTPS internal case management API. No individual case details or PHI are posted to chat webhooks (Slack/Teams).
3. **Resilient Retry Handling:** Exponential backoff (`1s`, `2s`) executes during transient network drops while failing fast if an unrecoverable HTTP status occurs.
4. **Audit Artifact Continuity:** The `dispatch` job runs conditionally on `always()` (unless `cancelled`), ensuring that output queues generated by the audit step are dispatched even when statutory violations or red flags are detected.